

Lab 09: Transaction Management – Exercise Answers

Question 1: In A2, after running tx_lab.py file again, what happened in ACCT table (BAL)?

After running `tx_lab.py` the second time (with the intentional SQL error `where d = 'B'`), the ACCT table BAL values remained unchanged — they stayed the same as before the second run.

Here is why: The program first executed the deposit to account A (`bal = bal + 100`), which succeeded. Then it attempted to execute the withdrawal from account B, but the SQL statement contained a typo — the column name `id` was misspelled as `d`. This caused a `mysql.connector.Error` exception. The program entered the `except` block and executed `conn.rollback()`, which reverted all changes made during this transaction, including the successful deposit to account A.

This demonstrates the **Atomicity** property of transactions: either all operations in the transaction are completed successfully and committed, or none of them take effect. Since the second operation failed, the entire transaction was rolled back, and the database remained in its previous consistent state.

Question 2: In A3, why Tx 2 can't or can see the updated value?

- **Step 3:** Tx 2 **cannot** see the updated BAL. At this point, Tx 1 has executed the UPDATE but has not yet committed. MySQL's default isolation level is **REPEATABLE READ**, which prevents Tx 2 from reading uncommitted data. Tx 2 still sees the original value.
- **Step 5:** Tx 2 **still cannot** see the updated BAL, even though Tx 1 has already committed in Step 4. This is because under REPEATABLE READ isolation, Tx 2's transaction is still active (it has not committed yet). Tx 2 continues to see a consistent snapshot of the data from when its own transaction started. This ensures repeatable reads within the same transaction.
- **Step 7:** Tx 2 **can now see** the updated BAL. Tx 2 committed its own transaction in Step 6, which ended its previous transaction. When Tx 2 executes the SELECT in Step 7, it starts a new transaction and obtains a fresh snapshot of the database, which now includes Tx 1's committed changes.

This behavior demonstrates how REPEATABLE READ isolation ensures data consistency within a transaction by using snapshot-based reads (MVCC — Multi-Version Concurrency Control).

Question 4: In B1 – Step 6, can Tx 2 see the updated BAL? Compare with the result in A3. Why?

Yes, in B1 Step 6, Tx 2 **can** see the updated BAL (A = 200), even though Tx 1 has **not yet committed** the update.

This is different from the result in A3 (Step 3), where Tx 2 could not see the uncommitted update.

The reason is that Tx 2's isolation level is set to **READ UNCOMMITTED** in B1. This is the lowest isolation level, which allows **dirty reads** — a transaction can read data that has been modified by another transaction but not yet committed. In contrast, A3 used the default **REPEATABLE READ** isolation level, which does not allow dirty reads.

The danger of READ UNCOMMITTED was demonstrated in Step 7, where Tx 1 executed a ROLLBACK. This means the value (BAL = 200) that Tx 2 read in Step 6 was never actually committed to the database — it was a temporary, invalid value. Tx 2 read "dirty" data that was later discarded.

Question 5: In B2 – Step 6, can Tx 2 see the updated BAL? Compare with the result in B1. Why? Also, can Tx 2 see the updated BAL after COMMIT? Why?

Step 6: Tx 2 **cannot** see the updated BAL. The value of A still appears as its original value, not 200.

Compared to B1, this is a different result. In B1 (READ UNCOMMITTED), Tx 2 could see the uncommitted update at the same step. In B2, the isolation level is **READ COMMITTED**, which **does not allow dirty reads**. Since Tx 1 has not committed its update at Step 6, Tx 2 cannot see it.

After COMMIT (Step 8/9): Tx 2 **can** see the updated BAL (A = 200). Once Tx 1 commits in Step 7, the change becomes permanent. Since READ COMMITTED allows reading any data that has been committed, Tx 2 can now see the updated value immediately, even within its current transaction.

Note: This is different from REPEATABLE READ (A3), where Tx 2 could not see the committed change until Tx 2 itself also committed and started a new transaction. READ COMMITTED provides a weaker consistency guarantee — it prevents dirty reads but allows **non-repeatable reads** (the same query can return different results within the same transaction if another transaction commits changes in between).

Question 6: In B3, check Step 7 and Step 10 of Tx 2. Before and after COMMIT in Tx 1, can Tx 2 read 'C' or not? Why?

- **Step 7 (Before Tx 1 COMMIT):** Tx 2 **cannot** read 'C'. At this point, Tx 1 has inserted the row ('C', 200) but has not committed. Under the REPEATABLE READ isolation level, Tx 2 cannot see uncommitted data from other transactions.
- **Step 10 (After Tx 1 COMMIT):** Tx 2 **still cannot** read 'C'. Even though Tx 1 has committed the insertion in Step 8, Tx 2's transaction is still active and continues to operate on the same

consistent snapshot it had when its transaction began. Under REPEATABLE READ in MySQL (InnoDB), the snapshot is taken at the time of the first read in the transaction. Therefore, any new rows inserted and committed by other transactions after this snapshot are not visible to Tx 2.

This demonstrates how MySQL's REPEATABLE READ isolation level **prevents phantom reads**. A phantom read would occur if Tx 2 ran the same query (`SELECT ID FROM ACCT WHERE BAL = 200`) twice and got different sets of rows — but REPEATABLE READ ensures the same result set is returned each time within the same transaction.

Tx 2 would only be able to see the new row 'C' after it commits its own transaction (ending the current snapshot) and starts a new transaction.

Summary Table: Isolation Level Behavior

Scenario	READ UNCOMMITTED	READ COMMITTED	REPEATABLE READ
Can read uncommitted data (dirty read)?	Yes	No	No
Can see committed changes immediately within same transaction?	Yes	Yes	No
Same query returns same result within same transaction?	No	No	Yes
Phantom rows visible within same transaction?	Yes	Yes	No (in MySQL InnoDB)